Data Studio — Build Spec

Oct 28, 2026 · @Gana

Overview

Build an agent-assisted studio where a user describes a data product in business language and the agent maps each requirement to governed source columns, with the user approving every change before it applies.

The three mockup screens (Intent, Build, Review) are the visual reference; press Play on each to click through. This spec defines the behavior behind them.

Core principles

- The agent proposes; the user decides. Nothing in the mapping grid changes without a user action.
- Every mapped value can be explained: score breakdown, evidence, and rejected alternatives.
- One chat thread per data product, visible in every layout.
- A product cannot be published until it passes validation.

In scope: adaptive layout, typed agent responses, proposal workflow, explainable scores, review queue, row-scoped chat, gated publish, exports, feedback capture.

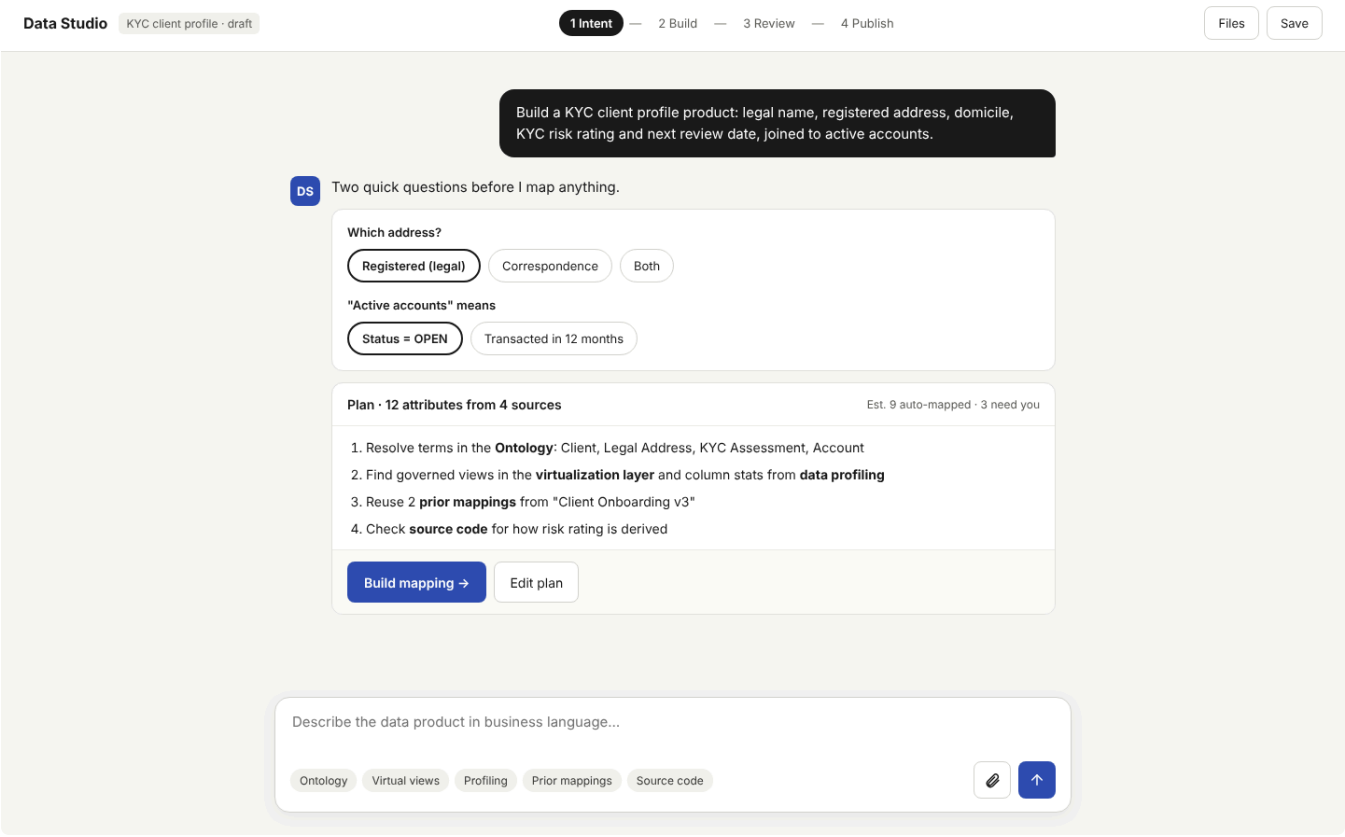
Out of scope for v1: multi-user real-time co-editing, building the physical pipeline, access-request workflows, and changes to the underlying source systems.

Source systems (generic names): Ontology (business terms), Virtual views (data virtualization layer), Profiling (column statistics), Prior mappings (approved past products), Source code (derivation logic). Map these to your actual systems.

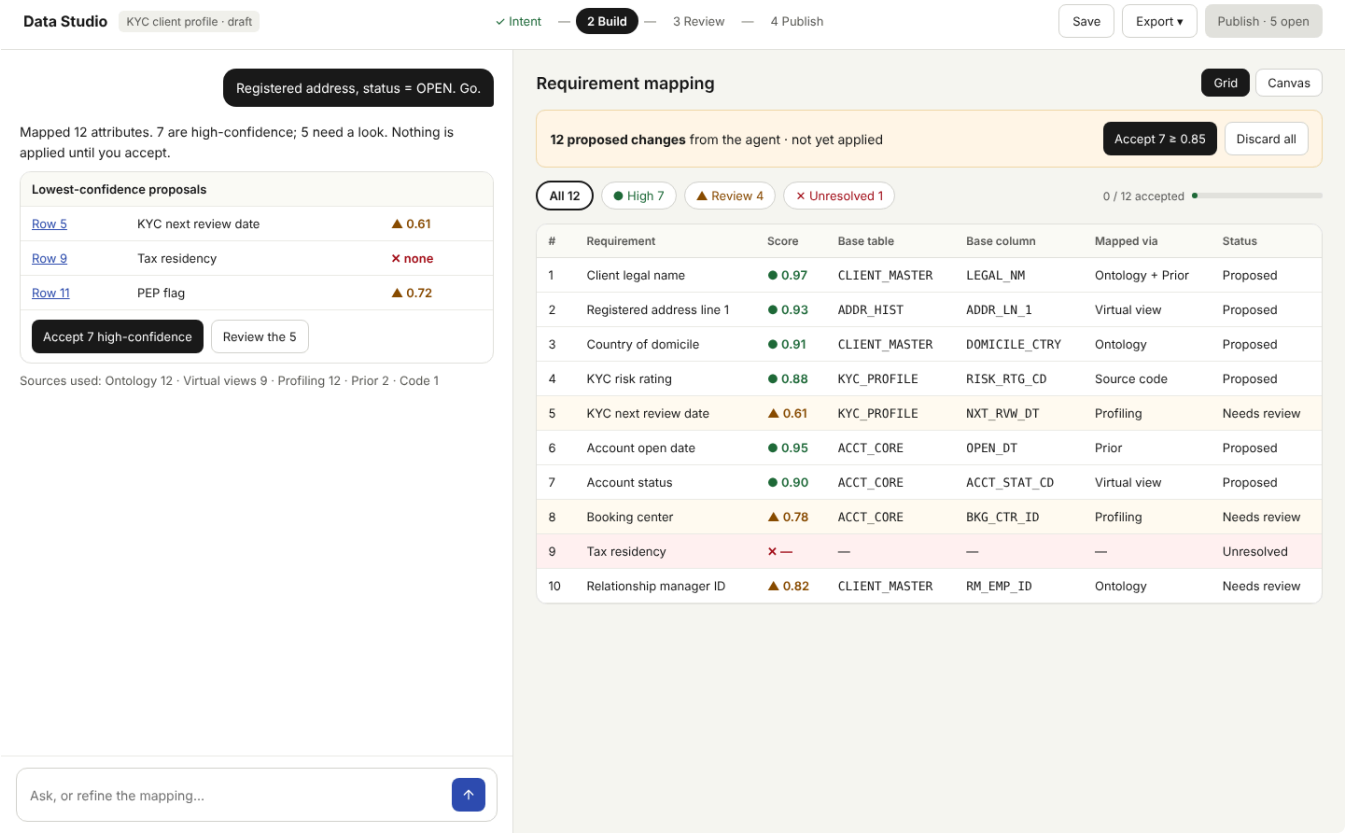
Screens

Four states of one session, using illustrative KYC data. Table and column names are placeholders.

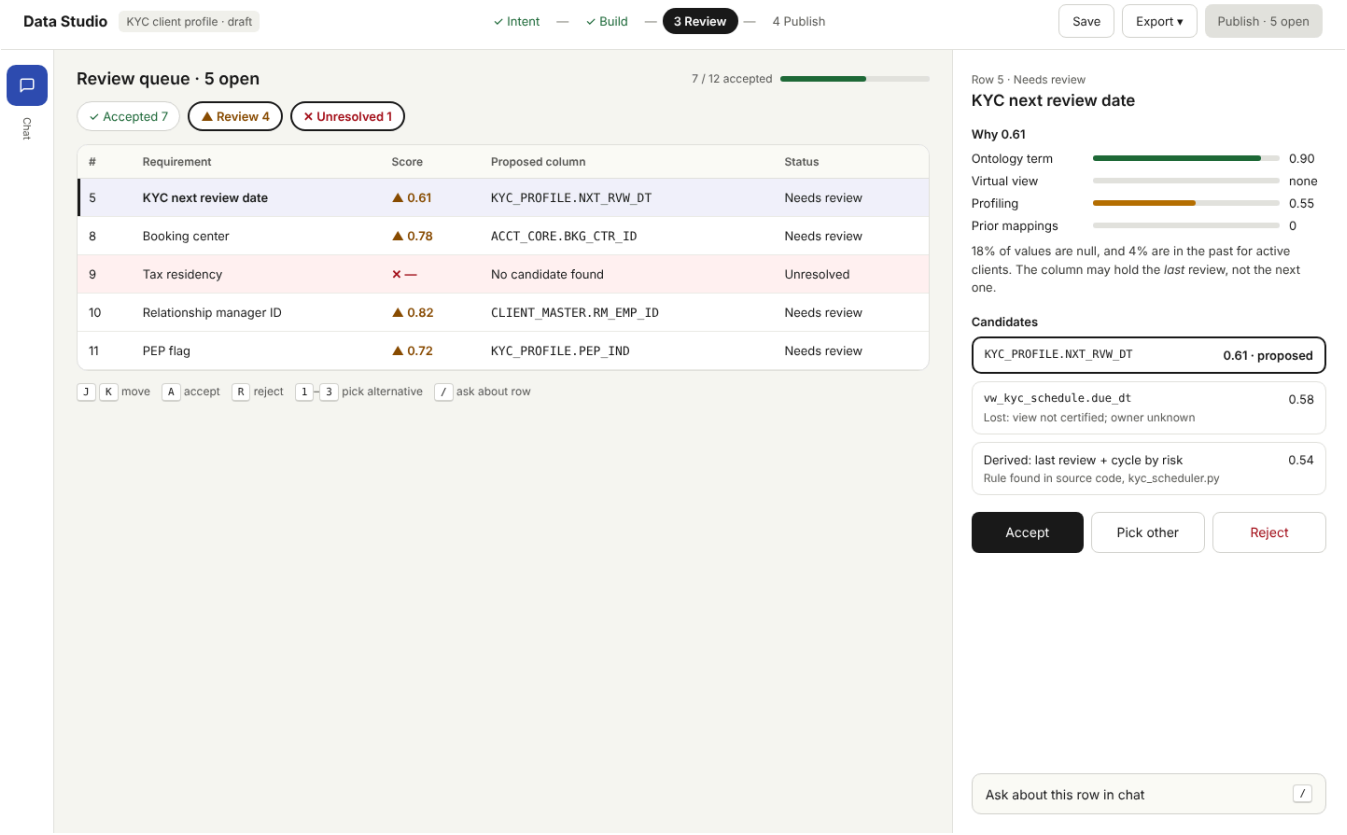
1 · Intent (chat-first). The agent asks clarifying questions as option chips and shows a plan before mapping anything.



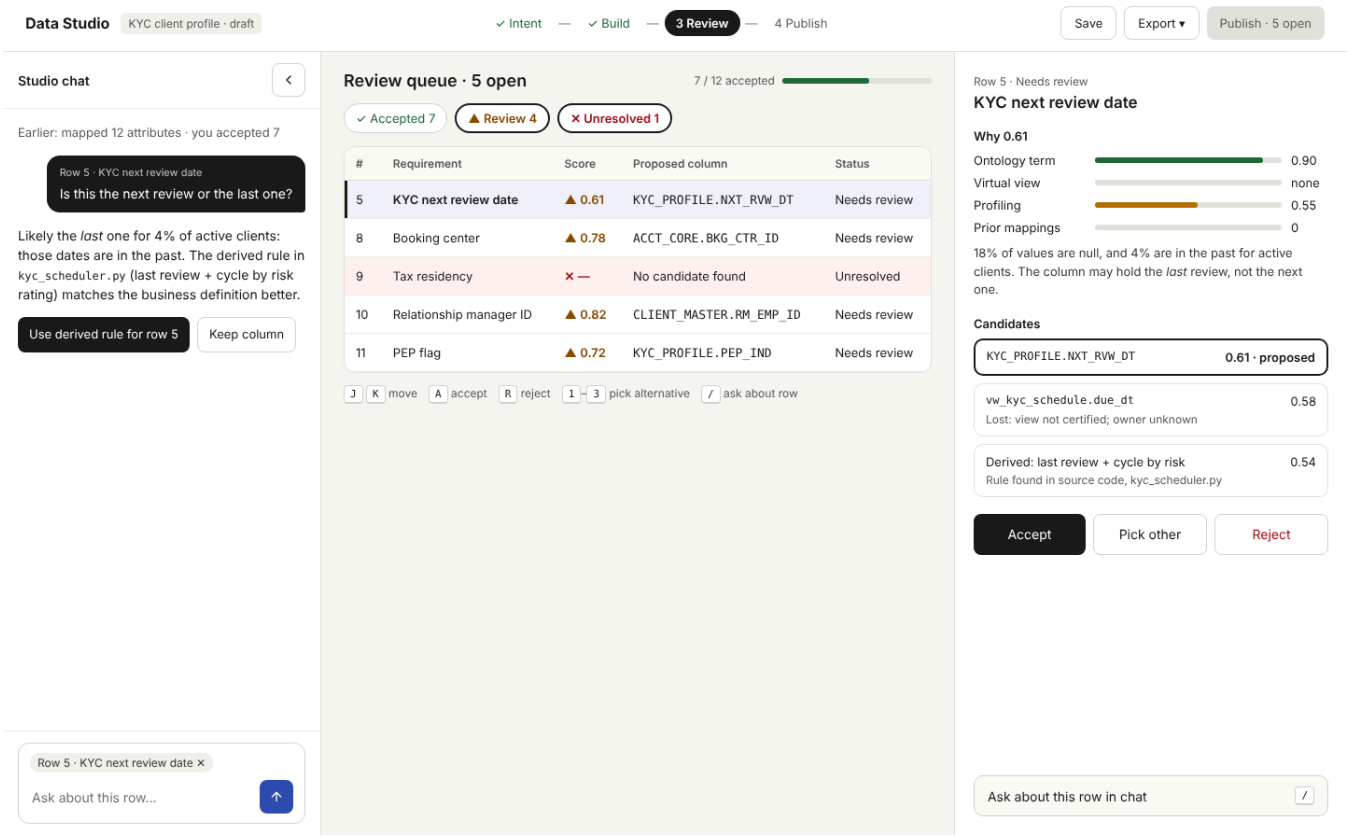
2 · Build (split view). Chat narrows; the grid shows 12 pending proposals with a banner, status filters and a progress bar. Publish is disabled while rows are open.



3 · Review (artifact-first). Chat collapses to a rail; the grid becomes a review queue, and the inspector explains the selected row's score and candidates.

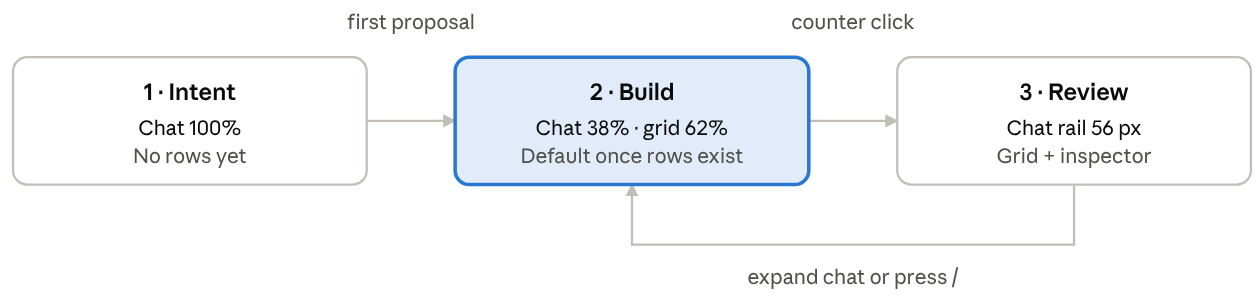


3b · Review with chat open. "Ask about this row" expands the same chat thread with the row attached; the answer ends in action buttons.



Layout states

The chat and grid share the screen in three modes, and the mode follows what the user is doing rather than a fixed split.



layout modes · 3 states, 3 transitions

A new product starts in Intent; the first proposal moves it to Build; clicking a counter moves to Review; expanding chat returns to Build.

Mode	Chat	Grid / artifact	Enter when
Intent	100% width, max 820 px column	Hidden	Product has 0 rows
Build	38% (min 340 px)	62%	First proposal arrives, or a session with rows is opened
Review	56 px rail	Grid plus 380 to 440 px inspector	User clicks a counter or a "Review" link

Rules

- The divider in Build is draggable; save the width per user in local storage.
- In Review, opening chat (rail icon, "Ask about this row", or /) expands it in place with the selected row attached. It is the same thread, not a second chat.
- Below 900 px width, panels stack vertically and the chat becomes a bottom sheet.
- A step indicator in the header (Intent, Build, Review, Publish) shows progress; completed steps are clickable.

Data model

Eight entities carry the whole workflow. Proposals are separate from accepted mappings, which is what makes review and audit possible.

Entity	Key fields	Notes
DataProduct	id, name, description, owner, status (draft / in_review / published), intent_summary, version	One per studio session
RequirementRow	id, product_id, row_no, requirement, required (bool), accepted_mapping_id, status (open / needs_review / unresolved / accepted / rejected)	Uploaded or created by the agent

Entity	Key fields	Notes
Mapping	id, row_id, base_table, base_column, derivation (nullable expression), mapped_via, score, score_breakdown (json)	One accepted mapping per row
Proposal	id, row_id, field, old_value, new_value, score, score_breakdown, evidence_ids[], candidate_ids[], reason, source_message_id, state (pending / accepted / rejected / superseded)	Agent output; never written to the grid directly
Candidate	id, row_id, target (table.column, view.column or derivation), score, rejected_reason	Alternatives the agent considered
Evidence	id, kind (ontology / virtual_view / profiling / prior_mapping / source_code), ref (term id, view name, file + line), summary, confidence	Shown in the inspector
ChatMessage	id, product_id, author (user / agent / system), row_id (nullable), blocks[] (typed, see Agent contract), created_at	One thread per product
Decision	id, proposal_id, user_id, action (accept / reject / pick_alternative / override), reason, created_at	Audit trail and learning signal

Rules

- A row is *accepted* only when a Decision accepts one of its proposals.
- Editing an accepted row creates a new proposal; the old mapping stays in history.
- A new proposal for the same row and field marks older pending ones *superseded*.

Agent contract

The agent returns a list of typed blocks per message, never free text alone. The UI renders each block as a component, and any action the user clicks becomes a Proposal or a Decision through the API.

Block type	Payload	Rendered as	Used in
text	markdown	Message body	All phases
clarify	question, options[], multi (bool)	Option chips; a pick is sent back as the user's answer	Intent
plan	steps[], sources[], est_auto, est_review	Plan card with Build and Edit plan buttons	Intent
mapping_preview	rows[] (row_id, requirement, score), total, high_confidence_count	Mini table plus Accept high-confidence and Review buttons	Build
proposal_batch	proposal_ids[]	Banner in the grid: N proposed, Accept, Discard	Build
row_answer	row_id, explanation, actions[]	Explanation plus action buttons	Review
action	label, kind (create_proposal / accept / reject / open_row), args	Button	Any
row_ref	row_id, label	Clickable chip that selects the row	Any

Contract rules

- The agent writes only Proposals, never Mappings. Accept and reject are user actions.
- Every proposal includes a score_breakdown and at least one evidence id, or it is rejected by the API.
- When the agent cannot find a candidate, it returns the row as *unresolved* with a reason, not a guess.

- Ask clarifying questions only when an ambiguity changes which columns are chosen; limit to 3 per turn.

Agent roles: a planner (intent to plan), a mapper (plan to proposals), and a checker (validates proposals against profiling and the ontology before they reach the user). Frameworks with typed tool output and shared UI state, such as AG-UI/CopilotKit, fit this contract.

Propose, review, apply

Agent output lands as pending proposals that the user accepts or rejects, one at a time or in bulk. This is the first thing to build; most other features depend on it.

Grid behavior

- A pending proposal shows its new value in the cell with a highlight, and the row status reads *Proposed* or *Needs review*.
- A banner above the grid shows the count of pending proposals, with "Accept N at score ⩾ 0.85" and "Discard all".
- Accepting writes a Decision, creates or updates the Mapping, and sets the row to *accepted*.
- Rejecting writes a Decision, keeps the previous mapping (if any), and moves the row back to *open* or *unresolved*.
- Undo is available for the last bulk action for 10 seconds, and any single decision can be reversed from row history.

API

- `POST /products/{id}/proposals` (agent only)
- `POST /proposals/{id}/accept`, `POST /proposals/{id}/reject`, `POST /proposals/{id}/pick-alternative` with `candidate_id`
- `POST /products/{id}/proposals/accept-bulk` with a filter (`min_score`, `row_ids`)
- `GET /rows/{id}/history`

Every call writes a Decision with user, time and optional reason.

Manual edits in the grid

When a user types directly in a grid cell, the agent must know, treat it as authoritative, and check it.

1. The edit is saved as a Decision with action *override* and the old and new values. It applies immediately; the user is the approver.
2. The checker agent validates the new value in the background (column exists, type and profiling match, ontology term fits) and returns a score breakdown with evidence.
3. A system event is posted to the chat thread, tagged with the row: "You set Row 5 base column to KYC_PROFILE.LAST_RVW_DT · checked: 0.88". If the check finds a problem, the event says why and offers actions (Keep mine, Use suggestion).
4. The agent never overwrites a user edit. Later proposals for that row and field are blocked unless the user asks the agent to revisit it.
5. The edit is included in the agent's context for later turns and stored as a labeled example for the learning loop.

API: `PATCH /rows/{id}` with field and value, which writes the Decision, triggers the check, and posts the chat event.

Score and evidence

Each score is a weighted sum of signals, and the inspector shows every signal so a reviewer can answer "why this column?" without asking the agent.

Signal	Meaning	Starting weight
Ontology term match	The requirement resolves to a business term bound to this column	0.35
Virtual view	A governed view exposes this column for the term	0.20
Profiling fit	Type, null rate, value range and freshness match the requirement	0.20
Prior mappings	Approved products used this mapping for the same term	0.15
Source code	Derivation logic found that produces this value	0.10

Weights are configuration, tuned against the evaluation set. A missing signal scores 0, not null.

Thresholds and display

- ⩾ 0.85: high (● green). Eligible for bulk accept.
- 0.60 to 0.85: needs review (▲ amber).
- < 0.60 or no candidate: unresolved (× red).
- Always pair color with the icon so status never depends on color alone.

Inspector (right panel in Review)

- Row name, status and score, then one bar per signal with its value.
- A one or two sentence plain-language finding from the checker, such as "18% of values are null; the column may hold the last review date."
- Candidates: the proposed one first, then alternatives, each with score and why it lost.
- Evidence links: ontology term, view name, profiling stats, prior product, file and line.
- Actions: Accept, Pick other, Reject, and Ask about this row in chat.

Review queue and keyboard

The status counters are filters that turn the grid into a to-do list, and a reviewer can clear it without a mouse.

- Counters above the grid: All, High, Review, Unresolved, Accepted. Clicking one filters the grid; Review and Unresolved can be combined.
- A progress bar shows accepted rows out of total rows.
- Selecting a row opens it in the inspector. After accept or reject, selection moves to the next open row.

Key	Action
J / K	Next / previous row
A	Accept the proposal
R	Reject the proposal
1 to 3	Pick alternative candidate 1 to 3
/	Open chat with this row attached
E	Edit the cell (manual override)
Esc	Close inspector or chat

Shortcuts are disabled while focus is in a text field. Show them in a footer hint and in a ? overlay.

Row-scoped chat

There is one chat thread per data product. In Review, asking about a row opens that same thread with the row attached, so every question, answer and decision stays in one history.

- **Row chip:** when a row is selected, the composer shows "Row 5 · KYC next review date ×". Messages sent with the chip store its `row_id`. Removing the chip makes the question product-wide.
- **Row tags in the thread:** user messages and system events show their row tag; clicking it selects the row in the grid.
- **This row only:** a toggle in the chat header hides messages whose `row_id` differs from the selected row. It is a client-side filter.
- **Answers end in actions:** a `row_answer` block with buttons such as "Use derived rule for row 5". Clicking creates a Proposal; it does not edit the grid.

Context sent to the agent for a row question

1. The row: requirement, current mapping, pending proposal, score breakdown, candidates, and evidence summaries.
2. Product-level decisions from the thread, such as clarify answers ("registered address", "status = OPEN").
3. User overrides and rejections on this row and on rows that share the same source table.
4. The last 10 messages tagged with this row.

Keep the bundle under a fixed token budget; drop older untagged messages first. Log every bundle so answers can be debugged.

Sizing: about 1 to 2 weeks for one developer once proposals exist. Most of the time goes into tuning the context bundle.

Publish, save and export

Publish stays disabled until validation passes, and the button says what is blocking it ("Publish · 5 open").

Validation rules (all must pass)

- [ ] No rows are *unresolved* or *needs review*; no pending proposals remain.
- [ ] Every required row has an accepted mapping.
- [ ] Product name, description and owner are set.
- [ ] Any accepted mapping below 0.60 has an override reason recorded.
- [ ] No accepted column fails a hard profiling check (wrong type, view not found).

Run validation on every change and show failures as a checklist in a Publish dialog, each item linking to the row or field to fix.

Header actions

- Save: secondary style, with autosave and a "Saved 12:09" timestamp.
- Export menu: mapping Excel, product JSON (your current catalog format), ODPS product descriptor, and ODCS data contract.
- Publish: primary style, gated as above.

Standards export: map product metadata (name, owner, description, version, status) to ODPS, and the schema, column descriptions and quality rules to ODCS. Validate the output with the open-source data contract CLI in CI.

Learning loop and evaluation

Every user decision is training signal: accepted mappings feed Prior mappings retrieval, and a fixed gold set measures whether changes to prompts, weights or models make mapping better or worse.

- **Capture:** each Decision (accept, reject, pick alternative, override) is stored with the requirement text, the chosen target, the rejected candidates and the reason.
- **Reuse:** accepted mappings from published products are indexed by ontology term and requirement text, and become the Prior mappings source.
- **Gold set:** start with 100+ rows from approved products. Each has the requirement and the correct target.
- **Metrics:** top-1 accuracy, top-3 accuracy, share of rows auto-accepted at ≥ 0.85, and false-high rate (rows scored ≥ 0.85 that a reviewer rejected).
- **Gate:** run the gold set in CI on every prompt, weight or model change; block merges that lower top-1 accuracy or raise the false-high rate.
- **Dashboard:** show the metrics over time so the team can show accuracy gains.

Build order and acceptance

Build trust first, then the richer experience, then learning. Each phase is shippable on its own.

Phase	Build	Done when
1	Data model, proposals API, grid highlights, accept and reject, Decision audit	Nothing in the grid changes without a user action, and every value has history
1	Score breakdown and inspector	A reviewer can explain any mapping from the inspector alone
1	Publish validation and gated button	An empty or incomplete product cannot be published
2	Adaptive layout (intent, build, review)	A new user never sees an empty grid; a returning user never sees an empty chat
2	Typed agent blocks (clarify, plan, mapping_preview, row_answer)	The agent never returns mappings as plain text
2	Row-scoped chat and manual-edit events	A question with a row chip is answered about that row; a grid edit appears in the thread and is checked
3	Review queue filters and keyboard	50 rows can be reviewed without a mouse
3	Learning loop, gold set, CI gate	Accuracy is reported for every release
3	ODPS and ODCS export	Exported contracts pass the data contract CLI

Open questions

- What score threshold should bulk accept use: 0.85, or configurable per product?
- Who can publish: the product owner only, or any reviewer?
- Should manual overrides below 0.60 require a second approver?
- Which existing systems back each source (ontology, virtual views, profiling, code search)?